RESUMEN DE PAPER · RESOLUCIÓN DE RESTRICCIONES EN PCG

WaveFunctionCollapse es resolución de restricciones «en estado salvaje»

WaveFunctionCollapse is Constraint Solving in the Wild

Autores	Isaac Karth y Adam M. Smith (Department of Computational Media, University of California Santa Cruz)

Publicación	Proceedings of the 12th International Conference on the Foundations of Digital Games (FDG '17), Hyannis, MA, EE. UU., 14–17 de agosto de 2017. ACM, 10 páginas
Cita APA	Karth, I., & Smith, A. M. (2017). WaveFunctionCollapse is constraint solving in the wild. En *Proceedings of the 12th International Conference on the Foundations of Digital Games (FDG '17)* (Artículo 68). ACM. https://doi.org/10.1145/3102071.3110566
Palabras clave	Resolución de restricciones, generación procedural de contenido, síntesis de texturas

Resumen

El algoritmo **WaveFunctionCollapse (WFC)** de Maxim Gumin genera imágenes a partir de ejemplos y surge de la práctica artesanal de la generación procedural. WFC genera imágenes nuevas en el estilo de un ejemplo asegurando que **cada ventana local de la salida aparezca en algún lugar de la entrada**. En términos operativos, implementa una **búsqueda voraz sin retroceso** (*backtracking*).

El paper examina WFC como un caso de **métodos de resolución de restricciones**: rastrea su influencia explosiva en la comunidad de artistas técnicos, explica su funcionamiento con conceptos de la literatura de restricciones y analiza sus fortalezas mediante una implementación sustituta en **programación de conjuntos de respuestas** (ASP).

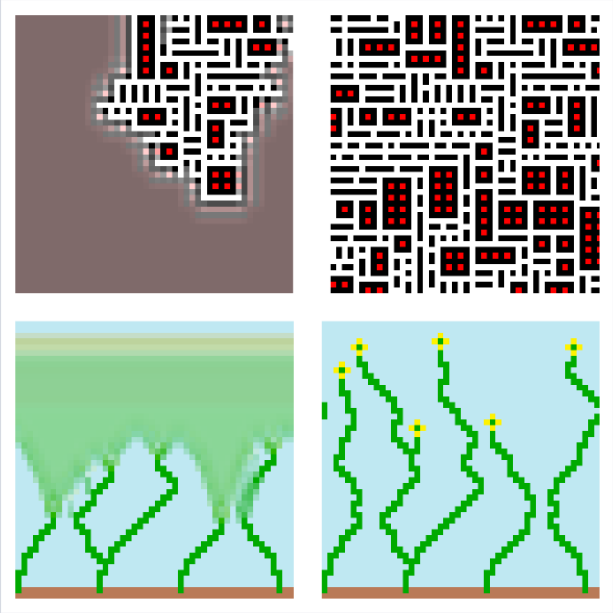


Figura 1. WaveFunctionCollapse en acción. La información aún no resuelta de las imágenes de la izquierda se muestra como el color promedio de sus salidas posibles.

1. Introducción

La resolución de restricciones no es un enfoque tradicional ni muy conocido en PCG, pero puede ser sorprendentemente eficaz para construir generadores controlables. En 2016 y 2017, WFC llamó la atención de desarrolladores indie y aficionados a través de redes sociales: las animaciones mostraban no solo el resultado (una imagen grande en el estilo de una pequeña), sino un modo de creación incremental sorprendentemente humano. Gumin escribe que, al dibujar, los humanos suelen seguir ellos mismos la heurística de **mínima entropía**.

WFC se apoya en técnicas de **síntesis de texturas** de la computación gráfica, pero se aparta de ellas en algo clave para el diseño de juegos: **no permite mezclar colores**, de modo que preserva la semántica de juego del ejemplo. En eso se parece a los métodos basados en cadenas de Markov, que arman salidas a partir de fragmentos localmente consistentes. Además, al ser un enfoque guiado por datos, es accesible para quienes no programan.

> **Contribuciones**
>
> - Un recorrido por las aplicaciones relevantes de WFC hasta la fecha.
> - Una explicación en pseudocódigo de los pasos principales del algoritmo, con un ejemplo y la terminología de la resolución de restricciones.
> - Una reformulación del problema en **ASP** para investigar de dónde vienen las fortalezas del algoritmo.

2. Antecedentes

2.1 Síntesis de texturas

La síntesis de texturas busca generar una imagen grande (a menudo repetible sin costuras) cuya textura se parezca a la de una imagen pequeña. Muchos métodos describen las imágenes por sus **patrones locales** (ventanas de pocos píxeles) y buscan que cada patrón de la salida se parezca a uno de la entrada; en gráficos ese parecido se mide con una distancia entre colores. En WFC, en cambio, el único parecido admitido es la **coincidencia exacta**.

Gumin se inspiró en la **síntesis discreta de modelos** de Paul Merrell, que genera modelos 3D grandes reutilizando exactamente trozos geométricos de un modelo pequeño. También probó la transferencia de estilo con redes neuronales (que no le sirvió para niveles) y proyectos propios: *SynTex* (buenas texturas, pero resultados sin sentido para objetos estructurados) y *ConvChain* (muestreo de Markov Chain Monte Carlo). Su objetivo era **capturar las reglas** con que se hizo la imagen de origen.

2.2 Algoritmos de resolución de restricciones

Los **problemas de satisfacción de restricciones** (CSP) se definen con **variables de decisión** y **valores**. En la generación de imágenes tipo WFC, hay **una variable por cada posición** de la salida, y sus valores son los **patrones locales únicos** de la imagen de entrada. Las **restricciones** indican qué combinaciones de valores son legales: aquí, que los patrones elegidos coincidan exactamente donde sus ventanas se solapan.

Un *solver* busca una **asignación total** que no viole restricciones. La mayoría busca en el espacio de **asignaciones parciales**: elige una variable sin asignar, le da un valor y, si llega a un callejón sin salida, **retrocede**. Las **heurísticas** ayudan a elegir qué variable y qué valor probar, sin afectar la completitud. La **propagación de restricciones** (p. ej. AC3) elimina de los dominios los valores que llevarían a callejones sin salida, ahorrando búsqueda.

2.3 Resolución de restricciones en PCG

Salvo excepciones, este enfoque ha sido ignorado en PCG; las taxonomías como la de PCG basada en búsqueda no contemplan métodos que no son constructivos ni buscan sobre diseños completos. Los métodos basados en restricciones suelen ofrecer **garantías fuertes** a costa de un tiempo de ejecución impredecible. Ejemplos: Horswill y Foged poblaron niveles con **garantías de jugabilidad** usando retroceso con propagación AC3; y en **Tanagra** (G. Smith), herramienta de diseño de niveles de **plataformas**, el solver Choco resuelve un subproblema de disposición geométrica y, si es insatisfacible, se relajan (retroceden) otras decisiones. **WFC, en cambio, no retrocede.**

2.4 ASP en PCG

La programación de conjuntos de respuestas (ASP) es una forma de programación lógica para modelar problemas de búsqueda y optimización combinatoria: las restricciones de bajo nivel se derivan de reglas de alto nivel, y el CSP resultante se resuelve con algoritmos del mundo SAT/SMT. A. M. Smith propuso usarla en PCG **modelando espacios de diseño**: describir declarativamente el contenido deseado y dejar lo procedimental al solver. Más adelante se usa Clingo para construir un sustituto experimental de WFC.

3. WFC «en estado salvaje»

Al día siguiente de su publicación (30 de septiembre de 2016), otros desarrolladores ya experimentaban con WFC, y pronto surgieron muchas reimplementaciones.

- **Joseph Parker** creó un conjunto de herramientas para Unity3D que aplicaba el sistema de patrones solapados a objetos 3D. Su juego **Proc Skater 2016** fue el **primero en usar WFC para generar niveles** (parques de skate), arreglando trozos geométricos discretos cuya coincidencia exacta aseguraba que el recorrido fuera fluido.
- **Oskar Stålberg** lo generalizó con otras formas de tile, 3D, mallas, optimizaciones y **retroceso**.
- **Caves of Qud** (Freehold Games), un *roguelike* comercial, lo usa en varias pasadas con plantillas sucesivas; su sencillez facilita que todo el equipo experimente.
- **Martin O'Leary** creó un generador de poesía: cada «tile» es una secuencia de sílabas en un WFC unidimensional, con restricciones de largo alcance (rima, métrica). Aquí la visión de WFC como resolución de restricciones es mucho más informativa que la de síntesis de texturas.

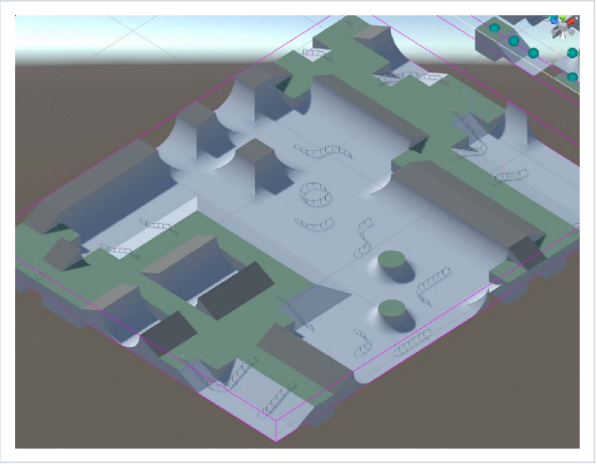


Figura 2. Generación de niveles en Proc Skater 2016, el primer juego que usó WFC.

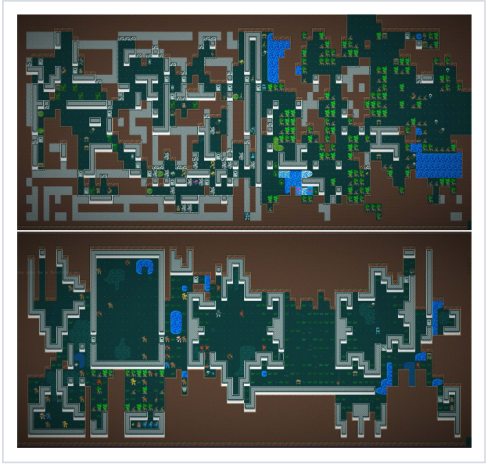


Figura 3. Dos niveles de sitios históricos en Caves of Qud generados con WFC.

4. El algoritmo WFC

El código original tiene menos de mil líneas en C#, pero es difícil de interpretar, y muchos usuarios lo trataban como caja negra. Los autores lo explican con el ejemplo **Red Maze**, una imagen de 4×4 píxeles y tres colores (Figura 4). A alto nivel, WFC hace cuatro cosas:

```
defn Run():
    PatternsFromSample()     # extrae los patrones locales de la entrada
    BuildPropagator()        # construye el índice de adyacencias legales
    Loop until finished:
        Observe()            # elige una celda y le asigna un patrón
        Propagate()          # elimina opciones incompatibles en los vecinos
    OutputObservations()     # convierte la asignación total en imagen
```

Un **patrón** es una configuración única de tiles de entrada. En la **versión de tiles simple**, los patrones y sus relaciones los especifica el diseñador; en la **versión de solapamiento**, se infieren de la imagen extrayendo todas las subimágenes únicas de N×N (opcionalmente con rotaciones y reflejos). Con N = 2, Red Maze tiene **12 patrones**; como el rojo y el blanco nunca están juntos en la entrada, no hay patrón con esa combinación.

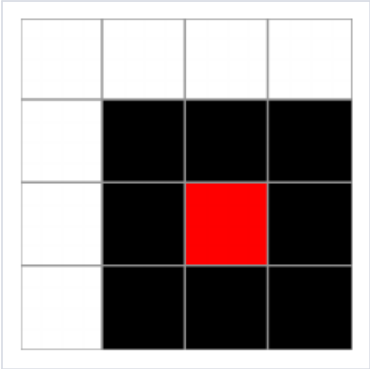


Figura 4. Red Maze: muestra de 4×4 píxeles (se repite periódicamente en los bordes).

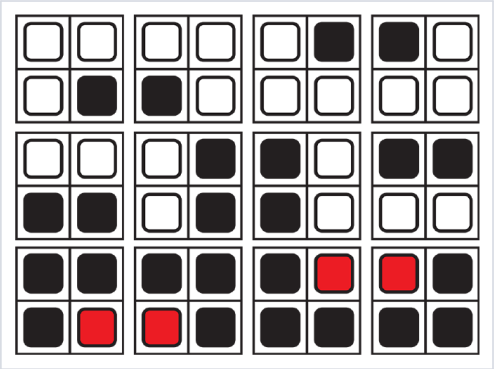


Figura 5. Los 12 patrones derivados de Red Maze con N = 2, reflexión y rotación. Solo aparecen patrones presentes en la muestra.

BuildPropagator() crea un **índice** que dice qué patrones pueden ubicarse junto a cuáles con cada desplazamiento (x, y). En la versión de solapamiento precalcula si dos patrones coinciden al superponerse; en la de tiles se toma directamente de las relaciones del diseñador. Es una relación **dispersa**: la mayoría de los patrones no pueden ir junto a la mayoría de los otros. Gumin llama a este índice *propagator*; los autores lo llaman índice para no confundirlo con algoritmos de propagación como AC3, que WFC implementa implícitamente.

Nota del resumen: el paper dice que con N = 3 hay $(2(N - 1) + 1)^2 = 36$ desplazamientos, pero esa fórmula da 25.

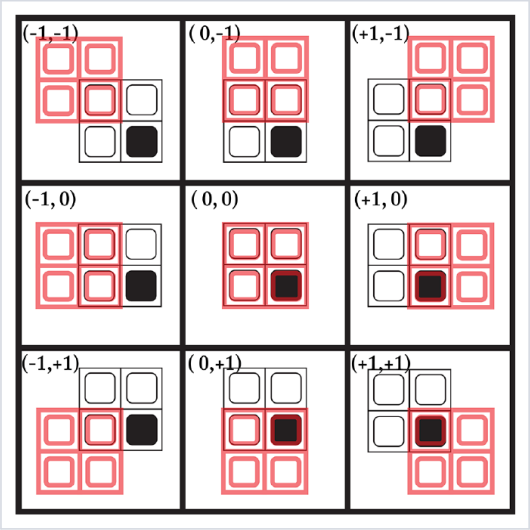


Figura 6. Las nueve formas en que dos patrones de 2×2 pueden solaparse.

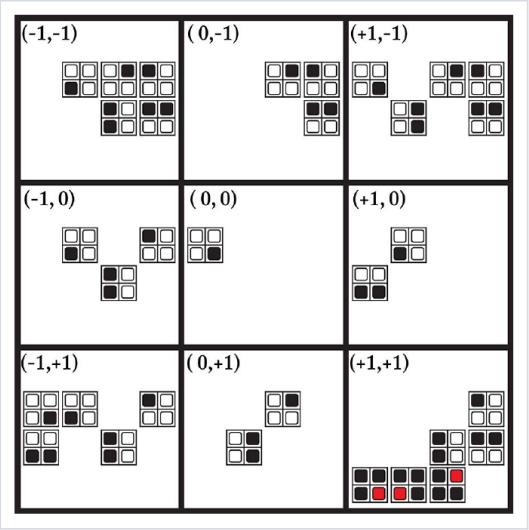


Figura 7. Porción del índice para el primer patrón de Red Maze: para cada desplazamiento, solo un subconjunto de los 12 patrones es válido.

Durante la generación se seleccionan y asignan variables (posiciones). Como todo solver, WFC lleva la cuenta de los **dominios restantes**: en el código se guarda en una tabla llamada **wave**, cuyos **coeficientes** booleanos indican si un patrón todavía puede ir en una posición. Todos empiezan en verdadero (dominio completo) y solo pueden pasar a falso. **No hay retroceso local:** si aparece un conflicto, el algoritmo **reinicia todo**.

```
defn Observe(coefficient_matrix):
    FindLowestEntropy()
    Si hay una contradicción: error y salir
    Si todas las celdas tienen entropía 0: terminar y devolver el resultado
    Si no: elegir un patrón al azar, ponderado por su frecuencia en la entrada,
           y marcar como falsos todos los demás patrones de esa celda
```

Observe() elige la posición con **menor entropía no nula**, es decir, la variable con el dominio más pequeño tras la propagación. Es la heurística clásica de **mínimos valores restantes** (MRV). Su justificación: si se aproxima el número de soluciones restantes como el producto de los tamaños de los dominios, asignar primero la variable con menos opciones maximiza las soluciones que siguen siendo posibles. El patrón se elige al azar según su **frecuencia** en la entrada, para que la salida conserve una distribución de patrones similar.

```
defn FindLowestEntropy(coefficient_matrix):
    Devolver la celda con la menor entropía mayor que cero, donde:
      una celda con un solo patrón válido tiene entropía 0
      una celda sin patrones válidos es una contradicción
      si no, la entropía depende de la suma de las frecuencias de los patrones
      (más algo de ruido aleatorio para desempatar)
```

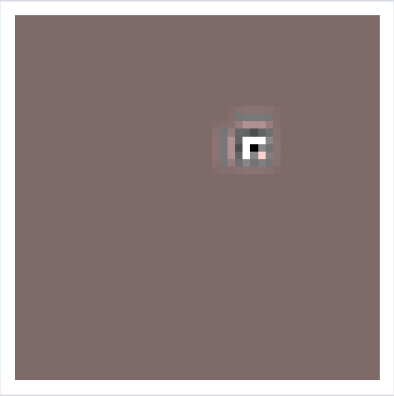


Figura 8. Primer paso de observación y propagación con Red Maze: la propagación ya resolvió dos píxeles blancos adicionales.

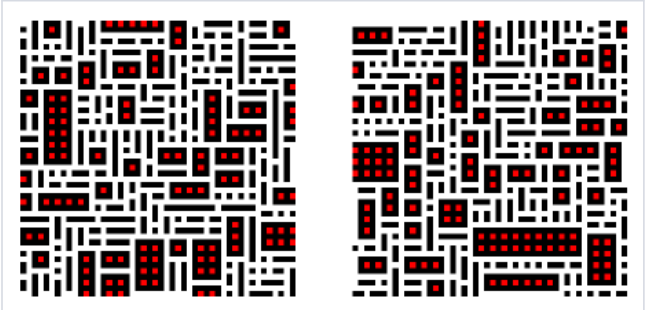


Figura 10. Dos de los resultados posibles con Red Maze.

Propagate() implementa **consistencia de arco**, como AC3: un valor solo permanece en el dominio de una variable si existe un valor compatible en las variables relacionadas. Como reducir un dominio puede afectar a los vecinos, la propagación avanza como un **relleno por inundación** (*flood fill*).

```
defn Propagate(coefficient_matrix):
    Repetir hasta que no queden celdas por actualizar:
        Para cada celda vecina:
            Para cada patrón aún válido:
                Comparar esa posición del patrón con los valores de la celda
                Si ya no coincide:
                    Marcar el patrón como falso en la wave
                    Marcar la celda para actualizar en la siguiente iteración
```

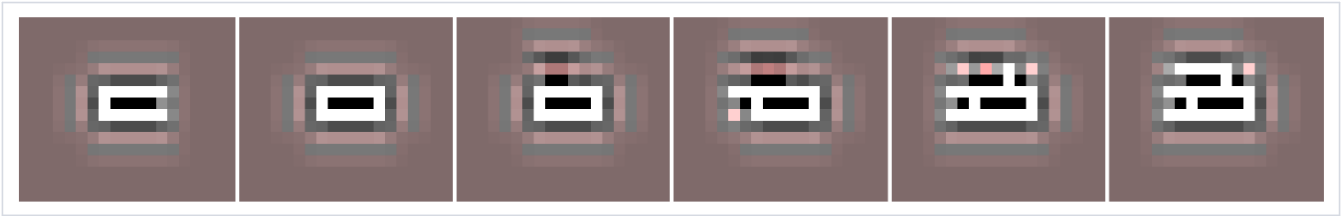


Figura 9. Actualizaciones consecutivas con Red Maze. Las celdas parcialmente resueltas se muestran como el promedio de sus salidas posibles; la propagación se expande más en algunos pasos.

Cuando todas las variables tienen un solo valor, se genera la imagen final; además, como cada celda guarda sus estados posibles, se pueden mostrar imágenes intermedias, lo que produjo las animaciones tan atractivas. En conjunto, **WFC es un algoritmo de resolución de restricciones**: usa MRV para elegir variable y la distribución de la entrada para elegir valor. Una alternativa sería la heurística de **valor menos restrictivo** (LCV), aunque es difícil predecir su efecto en la generación de contenido.

5. Reformulación de WFC en ASP

Se construye un **sustituto** de WFC en ASP. No reimplementa la heurística de entropía; busca capturar el **problema** que WFC resuelve para preguntarse cómo podría haberse implementado de otra forma. Reutiliza el procesamiento de entrada de Gumin y extrae el índice de adyacencias legales y la topología de la grilla. La formulación tiene solo **dos reglas**:

```
1 { assign(X,Y,P) : pattern(P) } 1 :- cell(X,Y).

:- adj(X1,Y1,X2,Y2,DX,DY),
   assign(X1,Y1,P1),
   not 1 { assign(X2,Y2,P2) : legal(DX,DY,P1,P2) }.
```

- **Regla 1:** a cada celda se le asigna **exactamente un patrón**.
- **Regla 2** (restricción de integridad): se rechaza una solución parcial si dos celdas adyacentes tienen patrones que el índice no marca como **compatibles**.

Solo se revisan los cuatro vecinos cardinales (|DX| + |DY| = 1), porque esa restricción incluye a las de mayor alcance. El tiempo de *grounding* es proporcional al número de celdas por el número de pares legales de patrones. Si se encuentra una solución sin retroceso, el tiempo de resolución es un polinomio modesto en el número de variables y restricciones; con retroceso, depende del número de conflictos. Los experimentos usan tres escenarios de Gumin con N = 3: **Platformer**, **Skyline** y **Flowers**.

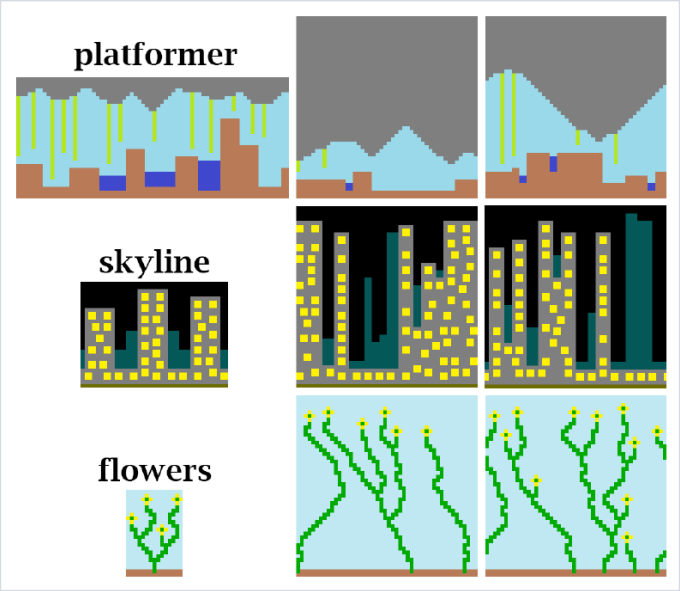


Figura 11. Tres imágenes de entrada (izquierda: platformer, skyline, flowers) con ejemplos de sus resultados (derecha).

5.1 Comprendiendo las heurísticas

Primero se ejecuta Clingo **sin heurísticas**, recorriendo la grilla en orden de lectura. Sorprendentemente, encuentra **cero conflictos** en los tres escenarios, incluso con elecciones aleatorias en cada posición. Lo mismo ocurre con **VSIDS**, la heurística por defecto de Clingo. Esto sugiere que **la fortaleza de WFC está en la propagación de restricciones**, no en la heurística de entropía. Gracias a la propagación, cada elección resuelve varias celdas: en vez de 48 × 48 = 2304 elecciones, se necesitan unas 325–433. En cambio, elegir la **celda siguiente al azar** es patológico: el solver siempre agota el minuto de tiempo tras miles de conflictos.

Tabla 1. Resultados de generar imágenes de 48×48 con Clingo 5.2.0 (10 imágenes por escenario). #Pat = patrones únicos de la entrada; #Var y #Con = variables y restricciones; #Choices = elecciones (asignaciones no hechas por propagación); #Conflicts = veces que hubo que deshacer una elección.

Escenario	#Pat	#Var	#Con	VSIDS #Choices	VSIDS #Conflicts	Sin heur. #Choices	Sin heur. #Conflicts
Platformer	116	486 238	1 976 634	390 ± 22	0	433 ± 45	0
Skyline	170	698 544	3 009 600	415 ± 14	0	411 ± 38	0
Flowers	106	485 958	1 909 342	325 ± 14	0	358 ± 34	0

5.2 Comprendiendo el retroceso

Los resultados anteriores explican por qué a Gumin le bastó con **reiniciar globalmente** ante un conflicto. Para ver qué tan frágil es esto, se agrega una restricción **global**: que **cada patrón de la entrada aparezca al menos una vez** en la salida.

```
:- pattern(P), not 1 { assign(X,Y,P) : cell(X,Y) }.
```

En Flowers y Platformer los conflictos casi no cambian, pero en **Skyline aparecen cientos**. Si Clingo reinicia globalmente tras cada conflicto (como WFC), **no encuentra solución en un minuto**; si se permite el **retroceso** (comportamiento por defecto de Clingo), la restricción se resuelve rápido ajustando decisiones locales.

Los autores anticipan que, al dar semántica de juego a los colores, crecerá la demanda de restricciones globales. Por ejemplo, para generar un entorno explorable sería deseable exigir **alcanzabilidad global**: **«todo lugar que el jugador pueda ocupar debe tener un camino factible desde la posición inicial»**. Caves of Qud resuelve hoy la conectividad con pasadas posteriores que agregan puertas y conexiones. Para satisfacer este tipo de restricciones de forma eficiente se necesitará **combinar retroceso local y reinicios globales**.

6. Conclusión

WFC es una aplicación importante de la resolución de restricciones en PCG, con múltiples usos reales. Como trabaja con trozos abstractos de contenido y no con colores mezclables, tiene aplicaciones como la poesía y la **generación de niveles con restricciones**. Los experimentos con el sustituto en ASP muestran que la heurística de entropía y los reinicios globales son decisiones razonables para la tarea original, pero **no son críticas**: otros (como Stålberg) ya están agregando retroceso local. Los autores afirman que **la búsqueda en el espacio de asignaciones parciales y la propagación de restricciones son las principales fortalezas de WFC**. Además, al ser un generador guiado por datos y con buen rendimiento, WFC desafía la idea de que los métodos de restricciones son poco accesibles: muchos artistas lo usan con éxito como caja negra.

> **Utilidad para tu investigación (plataformas 2D)**
>
> - **Formaliza WFC como CSP:** variables = posiciones de la grilla; dominios = patrones (o tiles) posibles; restricciones = compatibilidad entre vecinos. Calza con tu $X_{ij} \in T$.
> - **Muestra su límite clave:** WFC solo asegura restricciones **locales**; la jugabilidad (alcanzabilidad) es una restricción **global** que exige retroceso, como hace Sturgeon con un solver general.
> - **Relación con Launchpad:** WFC podría usarse para ensamblar piezas con reglas de compatibilidad, pero seguiría necesitando una verificación global de jugabilidad o un solver con retroceso.
> - **Detalle:** el escenario «Platformer» es una imagen de ejemplo de Gumin, no niveles jugables de un plataformas; el paper no evalúa jugabilidad.